

Practical: Addition of Two Large Vectors using CUDA C

Laboratory Practice V – Computer Engineering

1 Aim

Write a CUDA Program for Addition of two large vectors.

1.1 Objective

Students should be able to perform vector addition using CUDA C on GPU.

2 Theory

2.1 What is CUDA?

CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by **NVIDIA**. It allows developers to use the power of NVIDIA GPUs to accelerate computation.

2.2 What is Vector Addition?

Vector addition means adding corresponding elements of two arrays (vectors) to produce a third array.

$$C[i] = A[i] + B[i] \quad \text{for all } i = 0, 1, 2, \dots, n-1$$

Example:

Index	0	1	2	3	4	...
A[i] (= i)	0	1	2	3	4	...
B[i] (= i×2)	0	2	4	6	8	...
C[i] (= A+B)	0	3	6	9	12	...

2.3 How Vector Values are Generated

The code initializes vectors using formulas in a for loop:

```
for (int i = 0; i < n; i++) {
    h_A[i] = i;           // A = {0, 1, 2, 3, 4, ...}
    h_B[i] = i * 2;       // B = {0, 2, 4, 6, 8, ...}
}
```

First 10 values:

i	0	1	2	3	4	5	6	7	8	9
$A[i] = i$	0	1	2	3	4	5	6	7	8	9
$B[i] = i \times 2$	0	2	4	6	8	10	12	14	16	18
$C[i] = A+B$	0	3	6	9	12	15	18	21	24	27

Note: These are **not random** values. The for loop generates fixed values using formulas. Total 1024 elements in each vector ($n = 1024$).

2.4 Why Use GPU for Vector Addition?

Each element $C[i] = A[i] + B[i]$ is computed **independently**. This means we can assign each addition to a separate GPU thread.

```
CPU (Sequential):
  C[0]=A[0]+B[0] -> C[1]=A[1]+B[1] -> C[2]=A[2]+B[2] -> ...
  1024 steps, one after another

GPU (Parallel - 1024 threads):
  Thread 0: C[0]=A[0]+B[0]
  Thread 1: C[1]=A[1]+B[1]      ALL AT THE
  Thread 2: C[2]=A[2]+B[2]      SAME TIME!
  ...
  Thread 1023: C[1023]=A[1023]+B[1023]
```

2.5 CUDA Concepts Used

Term	Meaning
Host	CPU and its memory (RAM)
Device	GPU and its memory (VRAM)
Kernel	Function that runs on GPU (<code>__global__</code>)
<code>blockDim.x</code>	Number of threads per block (256)
<code>blockIdx.x</code>	Which block am I in?
<code>threadIdx.x</code>	Which thread am I within my block?
<code>cudaMalloc</code>	Allocate memory on GPU
<code>cudaMemcpy</code>	Copy data between CPU and GPU
<code>cudaFree</code>	Free GPU memory

2.6 Blocks and Threads

GPU organizes threads into **blocks**. Each block has a fixed number of threads.

```
n = 1024 elements
blockSize = 256 threads per block
gridSize = 1024 / 256 = 4 blocks

Block 0: Thread 0-255 -> handles C[0] to C[255]
Block 1: Thread 256-511 -> handles C[256] to C[511]
Block 2: Thread 512-767 -> handles C[512] to C[767]
Block 3: Thread 768-1023 -> handles C[768] to C[1023]

Total = 4 blocks x 256 threads = 1024 threads (one per element)
```

Thread index formula:

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

Example: Block 1, Thread 5:

$$i = 1 \times 256 + 5 = 261 \Rightarrow \text{this thread computes } C[261]$$

2.7 Program Execution Flow

```

Step 1: Initialize vectors A, B on CPU (h_A, h_B)
      |
Step 2: Allocate memory on GPU (cudaMalloc -> d_A, d_B, d_C)
      |
Step 3: Copy A, B from CPU -> GPU (cudaMemcpy HostToDevice)
      |
Step 4: Launch Kernel <<<4 blocks, 256 threads>>>
      (1024 threads compute C[i]=A[i]+B[i] simultaneously)
      |
Step 5: Copy result C from GPU -> CPU (cudaMemcpy DeviceToHost)
      |
Step 6: Print first 10 results
      |
Step 7: Free GPU memory (cudaFree)

```

3 Code

Listing 1: CUDA Vector Addition

```

1 #include <stdio.h>
2
3 // CUDA Kernel - each thread adds one element
4 // __global__ = runs on GPU, called from CPU
5 __global__ void vectorAdd(int *A, int *B, int *C, int n) {
6     // Calculate global thread index
7     int i = blockIdx.x * blockDim.x + threadIdx.x;
8
9     // Check if thread index is within array bounds
10    if (i < n) {
11        C[i] = A[i] + B[i];    // Each thread adds ONE element
12    }
13 }
14
15 int main() {
16     int n = 1024;                // Vector size: 1024 elements
17     int h_A[n], h_B[n], h_C[n]; // Host (CPU) vectors
18
19     // Step 1: Initialize vectors on CPU
20     for (int i = 0; i < n; i++) {
21         h_A[i] = i;              // A = {0, 1, 2, 3, ...}
22         h_B[i] = i * 2;          // B = {0, 2, 4, 6, ...}
23     }

```

```

24
25 // Device (GPU) pointers
26 int *d_A, *d_B, *d_C;
27
28 // Step 2: Allocate memory on GPU
29 cudaMalloc((void**)&d_A, n * sizeof(int));
30 cudaMalloc((void**)&d_B, n * sizeof(int));
31 cudaMalloc((void**)&d_C, n * sizeof(int));
32
33 // Step 3: Copy vectors from CPU to GPU
34 cudaMemcpy(d_A, h_A, n*sizeof(int), cudaMemcpyHostToDevice);
35 cudaMemcpy(d_B, h_B, n*sizeof(int), cudaMemcpyHostToDevice);
36
37 // Step 4: Launch kernel
38 int blockSize = 256; // 256 threads per
    block
39 int gridSize = (n + blockSize - 1) / blockSize; // = 4 blocks
40
41 vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);
42
43 // Step 5: Copy result from GPU to CPU
44 cudaMemcpy(h_C, d_C, n*sizeof(int), cudaMemcpyDeviceToHost);
45
46 // Step 6: Print first 10 results
47 printf("First 10 Results:\n");
48 for (int i = 0; i < 10; i++) {
49     printf("%d + %d = %d\n", h_A[i], h_B[i], h_C[i]);
50 }
51
52 // Step 7: Free GPU memory
53 cudaFree(d_A);
54 cudaFree(d_B);
55 cudaFree(d_C);
56
57 return 0;
58 }

```

4 Code Explanation

4.1 Kernel Function: vectorAdd()

- `__global__` — function runs on **GPU**, called from **CPU**.
- `i = blockIdx.x * blockDim.x + threadIdx.x` — calculates which element this thread should handle.
- `if (i < n)` — boundary check to avoid accessing memory beyond the array.
- `C[i] = A[i] + B[i]` — each thread adds **one** element.

4.2 Thread Index Calculation

blockIdx.x	threadIdx.x	$i = \text{block} \times 256 + \text{thread}$	Computes
0	0	$0 \times 256 + 0 = 0$	C[0]
0	1	$0 \times 256 + 1 = 1$	C[1]
0	255	$0 \times 256 + 255 = 255$	C[255]
1	0	$1 \times 256 + 0 = 256$	C[256]
1	5	$1 \times 256 + 5 = 261$	C[261]
3	255	$3 \times 256 + 255 = 1023$	C[1023]

4.3 Main Function Steps

Step	Function	Purpose
1	For loop initialization	Fill A and B with values
2	cudaMalloc()	Allocate memory on GPU
3	cudaMemcpy(HostToDevice)	Copy A, B from CPU → GPU
4	<<<4, 256>>>	Launch 1024 threads
5	cudaMemcpy(DeviceToHost)	Copy C from GPU → CPU
6	printf()	Print first 10 results
7	cudaFree()	Free GPU memory

4.4 Grid Size Calculation

```
int gridSize = (n + blockSize - 1) / blockSize;
              = (1024 + 256 - 1) / 256
              = 1279 / 256
              = 4 blocks
```

This formula ensures enough blocks to cover all elements, even if n is not perfectly divisible by `blockSize`.

5 Output

```
First 10 Results:
0 + 0 = 0
1 + 2 = 3
2 + 4 = 6
3 + 6 = 9
4 + 8 = 12
5 + 10 = 15
6 + 12 = 18
7 + 14 = 21
8 + 16 = 24
9 + 18 = 27
```

Verification:

- $A[3] = 3, B[3] = 3 \times 2 = 6, C[3] = 3 + 6 = 9$
- $A[9] = 9, B[9] = 9 \times 2 = 18, C[9] = 9 + 18 = 27$

6 How to Run

6.1 On Google Colab (Easiest – No GPU on PC needed)

1. Open `colab.research.google.com`
2. Runtime → Change runtime type → T4 GPU → Save
3. Cell 1:

Listing 2: Colab Cell 1: Write file

```
%%writefile vector_add.cu
#include <stdio.h>
__global__ void vectorAdd(int *A, int *B, int *C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) C[i] = A[i] + B[i];
}
int main() {
    int n = 1024;
    int h_A[n], h_B[n], h_C[n];
    for (int i = 0; i < n; i++) { h_A[i]=i; h_B[i]=i*2; }
    int *d_A, *d_B, *d_C;
    cudaMalloc((void**)&d_A, n*sizeof(int));
    cudaMalloc((void**)&d_B, n*sizeof(int));
    cudaMalloc((void**)&d_C, n*sizeof(int));
    cudaMemcpy(d_A, h_A, n*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, n*sizeof(int), cudaMemcpyHostToDevice);
    int blockSize=256, gridSize=(n+blockSize-1)/blockSize;
    vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);
    cudaMemcpy(h_C, d_C, n*sizeof(int), cudaMemcpyDeviceToHost);
    printf("First 10 Results:\n");
    for (int i=0; i<10; i++)
        printf("%d + %d = %d\n", h_A[i], h_B[i], h_C[i]);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}
```

4. Cell 2:

Listing 3: Colab Cell 2: Compile and Run

```
!nvcc vector_add.cu -o vector_add
!./vector_add
```

6.2 On Ubuntu (with NVIDIA GPU – College Lab)

1. Check if NVIDIA GPU is available:

```
nvidia-smi
```

If GPU info is displayed, proceed. If error, system has no NVIDIA GPU.

2. Install CUDA Toolkit (if not installed):

```
sudo apt update  
sudo apt install nvidia-cuda-toolkit
```

3. Verify installation:

```
nvcc --version
```

4. Create the CUDA file:

```
gedit vector_add.cu
```

Paste the code and save.

5. Compile:

```
nvcc vector_add.cu -o vector_add
```

6. Run:

```
./vector_add
```

7 Conclusion

In this practical, we implemented addition of two large vectors using CUDA C. The `__global__` kernel function assigned each thread to add one element independently using the formula $C[i] = A[i] + B[i]$. With 1024 elements distributed across 4 blocks of 256 threads each, all additions were performed simultaneously on the GPU. This demonstrates the massive parallelism capability of CUDA for simple element-wise operations on large datasets.